Lab 1 – Peer Assignment

In Lab 1, you are required to construct a Java desktop application that displays several important data items and a chart for an account dashboard (example dashboard in Figure 1).

The project is called *AccountDashboard* and the program structure is organised into three layers such as ***presentation*** layer, ***business logic*** layer and ***connection*** layer. The ***presentation*** layer will display the graph on GUI frame and call methods from ***ExpensesCompute* bean**. The ***business logic*** consists of algorithm to construct a chart. While, ***connection*** layer is to connect to the database. The business logic and connection layers require no main method. Example classes as below:

AccountDashboard
|- *MainWindow*
|- *ExpensesChart*
|- *SQLConnection*

The application must be construct using database, design pattern, component (i.e. ***ExpensesCompute* bean**) and program library (i.e. Chart library):

- ***ExpensesCompute* bean** is a pre-fabricated bean to calculate relevant spending information (Appendix 1)
- Database to store a dataset of year, month, expenses category and amount spend as shown in Table 1.
- Use ***Singleton*** [1] design pattern for the ***SQLConnection*** class to ensure only one instance of connection is established to the database. Refers Appendix 2.
- Use program library (e.g. ***JFreeChart*** [2], **XChart** [3] etc.) to generate graph to represents the dataset in Lab 1 by making query from Lab 1 repository. The graph must include chart title, label axis x and y.

Table 1. Dataset – Year, Month, Category & Expenses Amount

Year	Month	Category	Amount (integer type)	Year	Month	Category	Amount (integer type)
2026	January	Food & Beverages	450	2026	Mac	Food & Beverages	600
2026	January	Entertainment	100	2026	Mac	Entertainment	130
2026	January	Leisure & Sports	80	2026	Mac	Leisure & Sports	180
2026	January	Services	300	2026	Mac	Services	200
2026	January	Shopping	200	2026	Mac	Shopping	150
2026	January	Telecom	50	2026	Mac	Telecom	60
2026	January	Utilities	80	2026	Mac	Utilities	70
2026	February	Food & Beverages	390	…	…	…	…
2026	February	Entertainment	80	…	…	…	…
2026	February	Leisure & Sports	70	…	…	…	…
2026	February	Services	250	…	…	…	…
2026	February	Shopping	450	…	…	…	…
2026	February	Telecom	50	…	…	…	…
2026	February	Utilities	70	…	…	…	…

Submission Material in <u>ONE zipped folder, which label with your MATRIX No.</u>:

1. Sequence diagram
2. Program codes & execution files (*.jar) – zipped project folder
3. Export MySQL Database Schema (structure & data)
4. Screenshot of graphs

References:

1. Singleton Design Pattern, Retrieved from https://sourcemaking.com/design_patterns/singleton, March 2015
2. JFreeChart1.5.6, Retrieved from http://www.jfree.org/jfreechart/
3. XChart, Retrieved from https://knowm.org/open-source/xchart/

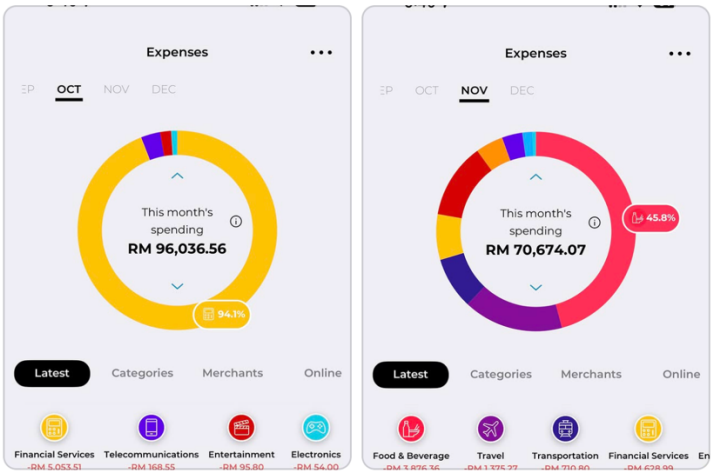

Figure 1. Example dashboard

Appendix 1: Construct *ExpensesCompute Bean*

1. Purpose

In Component-Based Software Engineering (CBSE), software components should be designed to be modular, reusable, and stateful. In line with this concept, this lab introduces the development of a non-visual JavaBean component for handling financial computation and persistent data storage.

For this lab, students are required to develop a JavaBean named *ExpensesCompute*. The component is expected to perform financial computations and maintain its data persistently by using serialization.

2. Objectives

At the end of this lab, students should be able to:

1. Develop a non-visual JavaBean following standard JavaBean conventions
2. Implement persistence using Serializable interface
3. Manage data using arrays or ArrayList
4. Generate documentation using Javadoc and package the component

3. Instructions

In this exercise, you are required to develop a JavaBean named:

com.ex.calculate.ExpensesCompute

The bean must:

- Have a no-argument constructor
- Implement the Serializable interface
- Import necessary packages
- Act as a non-visual component

3.1 Required Methods

1. **Yearly Total Expenses** → Sum of all expenses for the year
   Parameter: array of integers
   Return: integer
   Example: [100, 200, 300] → total = 600

2. **Monthly Total Expenses** → Total expenses for January, February, etc.
   Parameter: array of integers
   Return: integer
   Example: [100, 200, 300] → total = 600

3. **Yearly Total by Category** → Example: Total Food & Beverages in 2026
   Parameter: array of integers
   Return: integer
   Example: [100, 200, 300] → total = 600

4. **Average Monthly Expenses** → Total yearly / number of months
   Parameter: integer, integer
   Return: double
   Example: [1000, 4] → average monthly = 250

5. **Month-to-Month Comparison**
   → Difference between January and February
   → (Increase / Decrease)

6. **Percentage Change Between Months**
   → ((Feb – Jan) / Jan) × 100

Appendix 2: Singleton Design Pattern

Singleton pattern ensure a class has only one instance, and provide a global point of access to it. Figure 2 shows the ***Singleton*** design pattern structure.

- How does one ensure that only one instance of the class is created?
- Solution – restrict access to the constructor!

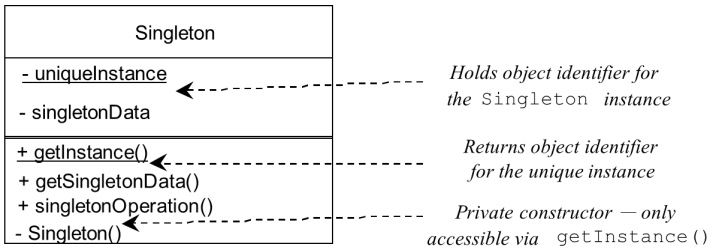

Figure 1. Singleton pattern

Figure 2 shows an example of ***Company*** class that use Singleton pattern. ***Company*** class have its constructor as private and have a static instance of itself. The ***Company*** class provides a static method to get its static instance to outside world, the method algorithm is shown in Table 1.

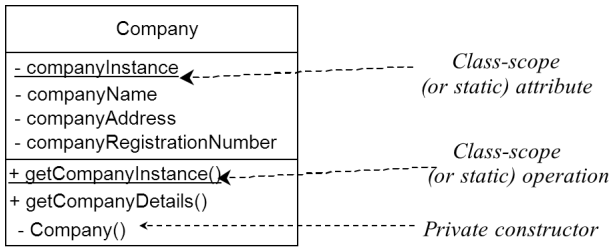

Figure 2. *Company* class using *Singleton* pattern

Table 1. *getInstance()* method algorithm

```
getCompanyInstance() :

        if (companyInstance == null)
          {
                companyInstance = new Company ()
          }
          return companyInstance
```

As for the ***SQLConnection*** class, Figure 3 shows the class design after adopting ***Singleton*** design pattern.

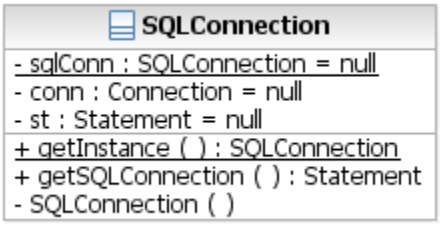

Figure 3. *SQLConnection* class using *Singleton* pattern

```
/**Sample Code for SQLConnection class*/

public class SQLConnection {

    private Connection con = null;
    private Statement st = null;

    /** Creates a new instance of SQLConnection */

    public Statement getSQLConnection () {
        String driver = "com.mysql.jdbc.Driver";
        String url = "jdbc:mysql://localhost:3306/";
        String dbname = "lab";
        String username = "root";
        String pwd = "";

        try {
          Class.forName(driver);

          con = DriverManager.getConnection(url + dbname, username, pwd);
          st = con.createStatement();

        }  //try
        catch (Exception e) {
           System.out.println(e);
        }    //catch

        return st;
    }

}
```